

Motor de Búsqueda Distribuido con MPI, Ranking TF-IDF y Balance de Carga

Sebastian Alzate Vargas – COMP6786 – Proyecto Final

Resumen

Este proyecto implementa un motor de búsqueda distribuido utilizando MPI. El sistema trabaja con una colección grande de documentos y permite buscar una consulta dentro de ellos. Se desarrollaron varias versiones del programa: una versión serial, una versión MPI simple, una versión MPI con ranking TF-IDF y una versión MPI TF-IDF con balanceo de carga por cantidad de palabras.

El objetivo principal fue estudiar cómo el diseño paralelo afecta el tiempo de ejecución, el speedup, la eficiencia y el balance de carga. Para esto se usó un conjunto de 100,000 documentos sintéticos y se compararon los resultados usando 1, 2, 4 y 8 procesos MPI. Los resultados muestran que MPI reduce el tiempo de búsqueda y que el diseño de la partición influye directamente en el rendimiento.

1. Introducción

El propósito de este proyecto fue construir un sistema distribuido sencillo, pero suficientemente completo, para analizar el rendimiento de una búsqueda de documentos usando MPI.

La idea general es parecida a un motor de búsqueda básico. Tenemos muchos documentos y una consulta, por ejemplo:

`mpi_speedup`

El programa debe revisar los documentos, calcular cuáles son los más relevantes para esa consulta y devolver los mejores resultados.

Si una sola computadora o un solo proceso revisa todos los documentos, el trabajo puede tardar más. Por eso se usa MPI: se divide el conjunto de documentos entre varios procesos. Cada proceso revisa una parte, calcula sus mejores resultados locales y luego el proceso principal combina todo para obtener el Top 10 global.

Este proyecto no solo busca que el programa corra más rápido. También busca mostrar cómo diferentes decisiones de diseño afectan el rendimiento. Por eso se implementaron varias versiones:

- una versión serial,
- una versión MPI simple,
- una versión MPI con ranking TF-IDF,
- una versión MPI TF-IDF con balanceo de carga.

2. Descripción del problema

El problema consiste en buscar una consulta dentro de una colección grande de documentos. Cada documento tiene un título, un tema, un número de palabras y un texto.

Por ejemplo, un documento puede tener esta información:

Campo	Ejemplo
ID	43690
Título	Parallel Computing Document 43690
Tema	parallel computing
Cantidad de palabras	156
Texto	Documento generado con palabras relacionadas al tema

El programa recibe una consulta y debe calcular un puntaje para cada documento. Los documentos con mayor puntaje se muestran como los más relevantes.

Este problema se puede paralelizar porque cada documento puede evaluarse de forma independiente. Es decir, un proceso puede revisar unos documentos mientras otro proceso revisa otros documentos. Después solo se necesita combinar los mejores resultados.

3. Dataset

El dataset se generó de forma sintética usando Python. Se decidió generar los documentos en vez de descargar un dataset externo para tener control sobre el tamaño de los datos y sobre los temas incluidos.

Cada documento contiene:

- ID del documento,
- título,
- tema,
- cantidad de palabras,

- texto completo.

Los temas usados fueron:

- parallel computing,
- distributed systems,
- machine learning,
- databases,
- cybersecurity,
- operating systems,
- data mining,
- sports analytics.

Para los experimentos principales se generaron 100,000 documentos. Además, los documentos no tienen todos el mismo tamaño. Algunos documentos son cortos y otros son largos. Esto permite analizar el balance de carga, porque repartir la misma cantidad de documentos no siempre significa repartir la misma cantidad de trabajo.

4. Idea general del sistema

El sistema funciona en varias etapas:

1. Se genera el dataset de documentos.
2. El usuario escribe una consulta.
3. El programa divide los documentos entre los procesos MPI.
4. Cada proceso busca la consulta en sus documentos.
5. Cada proceso calcula sus mejores 10 resultados locales.
6. El proceso 0 recibe los resultados locales.
7. El proceso 0 construye el Top 10 global.

La Figura 1 resume la idea general.

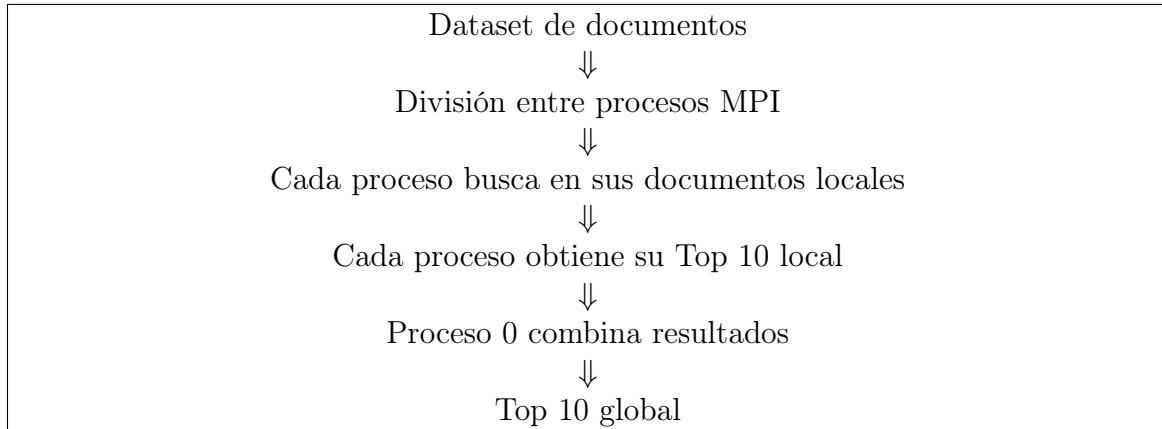


Figura 1: Flujo general del motor de búsqueda distribuido.

5. Versiones implementadas

5.1. Versión serial

La versión serial sirve como punto de comparación. En esta versión, un solo proceso lee todos los documentos y calcula el puntaje de cada uno.

El puntaje simple se calcula así:

$$\text{score} = 3 \cdot \text{apariciones en título} + \text{apariciones en texto}$$

El título tiene más peso porque normalmente resume mejor el contenido del documento. Por eso una coincidencia en el título vale tres veces más que una coincidencia en el texto.

Esta versión es fácil de entender, pero no aprovecha paralelismo.

5.2. Versión MPI simple

En la versión MPI simple, los documentos se dividen entre varios procesos. Si hay 100,000 documentos y 4 procesos, cada proceso recibe aproximadamente 25,000 documentos.

Cada proceso hace lo siguiente:

1. revisa sus documentos locales,
2. calcula el puntaje simple para cada documento,
3. guarda sus mejores 10 resultados,

4. envía esos resultados al proceso 0.

El proceso 0 junta los resultados de todos los procesos y obtiene el Top 10 global.

Esta versión es rápida porque solo necesita una pasada por los documentos.

5.3. Versión MPI con TF-IDF

La versión MPI simple cuenta apariciones de palabras. Eso funciona, pero es un ranking básico. Para mejorar el ranking se implementó TF-IDF.

TF-IDF es una técnica común en recuperación de información. La idea es que una palabra debe pesar más si aparece en un documento, pero no aparece en todos los documentos. Una palabra muy común no debería ser tan importante como una palabra más específica.

El IDF se calculó con la fórmula:

$$\text{idf}(t) = \log \left(\frac{N + 1}{df(t) + 1} \right) + 1$$

donde:

- N es el número total de documentos,
- $df(t)$ es el número de documentos donde aparece el término t .

Luego, el puntaje TF-IDF de un documento se calcula como:

$$\text{score} = \sum_{t \in \text{query}} (3 \cdot tf_{\text{titulo}}(t) + tf_{\text{texto}}(t)) \cdot \text{idf}(t)$$

Esta versión es más costosa porque necesita dos pasos:

1. calcular en cuántos documentos aparece cada término de la consulta,
2. calcular el puntaje TF-IDF de cada documento.

Para obtener el valor global de $df(t)$, cada proceso calcula su conteo local y luego se usa:

`MPI_Allreduce`

Esto permite sumar los conteos locales de todos los procesos y obtener una estadística global correcta.

5.4. Versión MPI TF-IDF balanceada

La versión TF-IDF normal reparte los documentos por cantidad de documentos. Por ejemplo, con 4 procesos cada proceso recibe cerca de 25,000 documentos.

El problema es que no todos los documentos tienen el mismo tamaño. Un proceso puede recibir 25,000 documentos cortos y otro proceso puede recibir 25,000 documentos más largos. En ese caso, ambos tienen la misma cantidad de documentos, pero no la misma cantidad de trabajo.

Para mejorar esto, se implementó una versión balanceada por cantidad de palabras. Esta versión usa el campo `WordCount` para intentar que cada proceso reciba una cantidad parecida de palabras.

La idea es repartir el trabajo así:

Partición estática	Misma cantidad de documentos
Partición balanceada	Cantidad parecida de palabras

Esto permite estudiar si una mejor distribución del trabajo mejora el tiempo de ejecución y el balance de carga.

6. Diseño paralelo con MPI

El diseño paralelo usa varios conceptos importantes de MPI.

6.1. Distribución del trabajo

Cada proceso recibe un rango de documentos. Por ejemplo, con 4 procesos:

Proceso	Documentos asignados
0	primera parte del dataset
1	segunda parte del dataset
2	tercera parte del dataset
3	cuarta parte del dataset

Cada proceso trabaja de forma independiente sobre su parte.

6.2. Comunicación global

En la versión TF-IDF, los procesos necesitan compartir información. Cada proceso calcula cuántos documentos locales contienen cada término de la consulta. Pero para calcular IDF

se necesita el conteo global.

Por eso se usa `MPI_Allreduce`. Esta operación suma los conteos locales de todos los procesos y entrega el resultado global a todos.

6.3. Combinación de resultados

Cada proceso mantiene un Top 10 local. Luego, el proceso 0 recoge esos resultados locales y construye el Top 10 global.

Esto reduce la cantidad de comunicación. En vez de enviar todos los documentos al proceso 0, cada proceso solo envía sus mejores resultados.

7. Métricas usadas

Para evaluar el proyecto se usaron varias métricas.

7.1. Tiempo de ejecución

Es el tiempo total que tarda el programa en completar la búsqueda.

7.2. Speedup

El speedup mide cuánto mejora el tiempo al usar varios procesos:

$$S_p = \frac{T_1}{T_p}$$

donde T_1 es el tiempo con 1 proceso y T_p es el tiempo con p procesos.

7.3. Eficiencia

La eficiencia mide qué tan bien se aprovechan los procesos:

$$E_p = \frac{S_p}{p}$$

Una eficiencia cercana a 1 significa que los procesos se están aprovechando bien. Si la eficiencia baja, puede ser por comunicación, sincronización, lectura de archivo, overhead o desbalance de carga.

7.4. Balance de carga

Para medir el balance de carga se usaron dos razones:

$$\text{word imbalance} = \frac{\text{máx}(\text{palabras por proceso})}{\text{mín}(\text{palabras por proceso})}$$

$$\text{time imbalance} = \frac{\text{máx}(\text{tiempo por proceso})}{\text{mín}(\text{tiempo por proceso})}$$

Un valor cercano a 1 significa mejor balance.

8. Resultados experimentales

Los experimentos se realizaron con 100,000 documentos y la consulta:

`mpi_speedup`

8.1. Resultados de MPI simple

Modelo	Procesos	Tiempo (s)	Speedup	Eficiencia
MPI Simple	1	1.002666	1.0000	1.0000
MPI Simple	2	0.532996	1.8812	0.9406
MPI Simple	4	0.314630	3.1868	0.7967
MPI Simple	8	0.244280	4.1046	0.5131

Tabla 1: Resultados de rendimiento para la versión MPI simple.

La versión MPI simple redujo el tiempo de 1.002666 segundos con 1 proceso a 0.244280 segundos con 8 procesos. Esto representa un speedup de 4.1046.

8.2. Resultados de MPI TF-IDF

Modelo	Procesos	Tiempo (s)	Speedup	Eficiencia
MPI TF-IDF	1	3.809188	1.0000	1.0000
MPI TF-IDF	2	1.818630	2.0945	1.0473
MPI TF-IDF	4	0.973448	3.9131	0.9783
MPI TF-IDF	8	0.801135	4.7547	0.5943

Tabla 2: Resultados de rendimiento para la versión MPI TF-IDF.

La versión TF-IDF tarda más que la versión simple porque hace más trabajo. Sin embargo, también se beneficia del paralelismo. El tiempo bajó de 3.809188 segundos con 1 proceso a 0.801135 segundos con 8 procesos.

La eficiencia con 2 procesos fue ligeramente mayor que 1. Esto puede ocurrir por efectos de caché, lectura de archivo o variación del sistema. No se interpreta como escalabilidad perfecta, sino como un resultado experimental influenciado por la máquina y el ambiente de ejecución.

9. Comparación visual de rendimiento

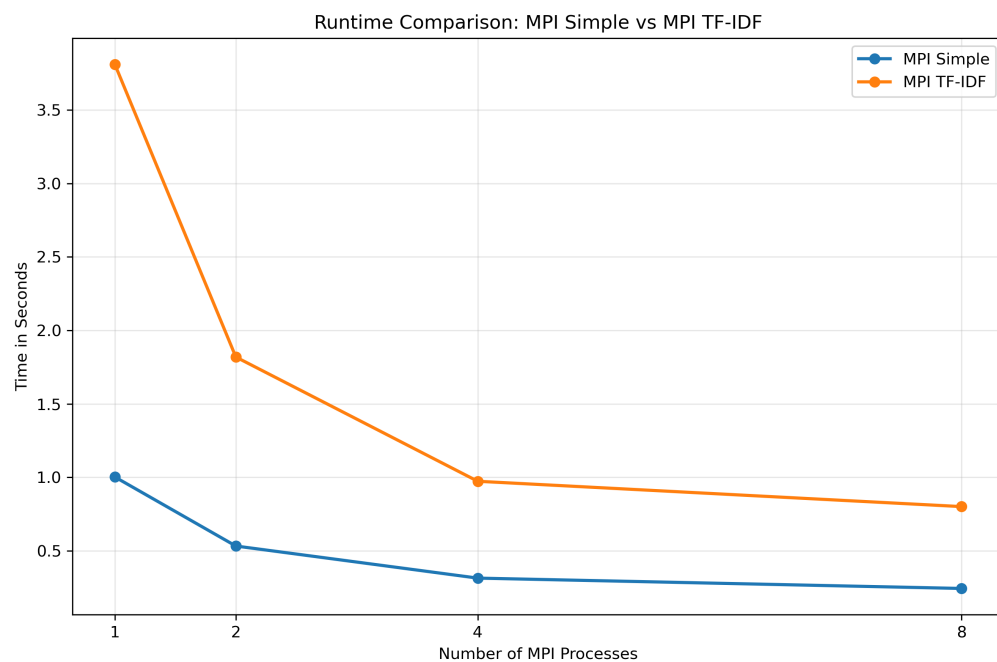


Figura 2: Comparación del tiempo de ejecución entre MPI simple y MPI TF-IDF.

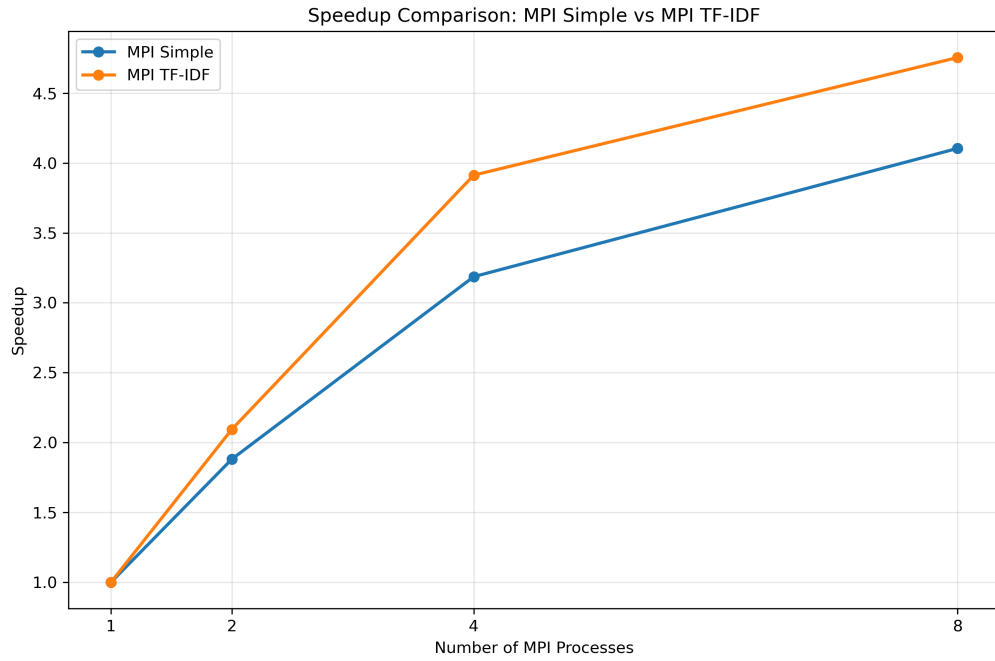


Figura 3: Comparación del speedup entre MPI simple y MPI TF-IDF.

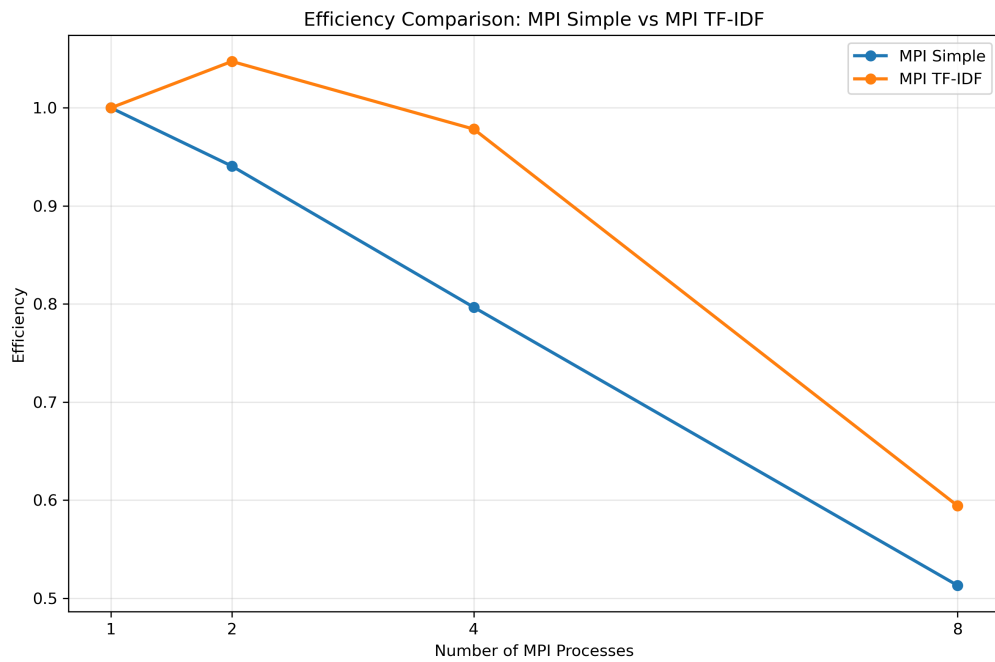


Figura 4: Comparación de eficiencia entre MPI simple y MPI TF-IDF.

10. Balance de carga

Versión	Procesos	Tiempo (s)	Speedup	Eficiencia	Word Imbalance	Time Imbalance
TF-IDF Static	4	0.973448	3.9131	0.9783	1.0233	1.0656
TF-IDF Balanced	4	0.916161	4.1578	1.0394	1.0001	1.0282
TF-IDF Static	8	0.801135	4.7547	0.5943	1.0286	1.1519
TF-IDF Balanced	8	0.795925	4.7859	0.5982	1.0009	1.2004

Tabla 3: Comparación entre partición estática y partición balanceada por palabras.

Con 4 procesos, la versión balanceada mejoró el balance de palabras y también redujo el tiempo total. El desbalance de palabras bajó de 1.0233 a 1.0001, y el tiempo bajó de 0.973448 a 0.916161 segundos.

Con 8 procesos, el balance de palabras también mejoró, bajando de 1.0286 a 1.0009. Sin embargo, el desbalance de tiempo aumentó ligeramente. Esto muestra que repartir palabras de forma más uniforme no garantiza tiempos exactamente iguales. También influyen otros factores como lectura de archivo, caché, overhead, sistema operativo y ubicación de los datos.

11. Gráficas de balance

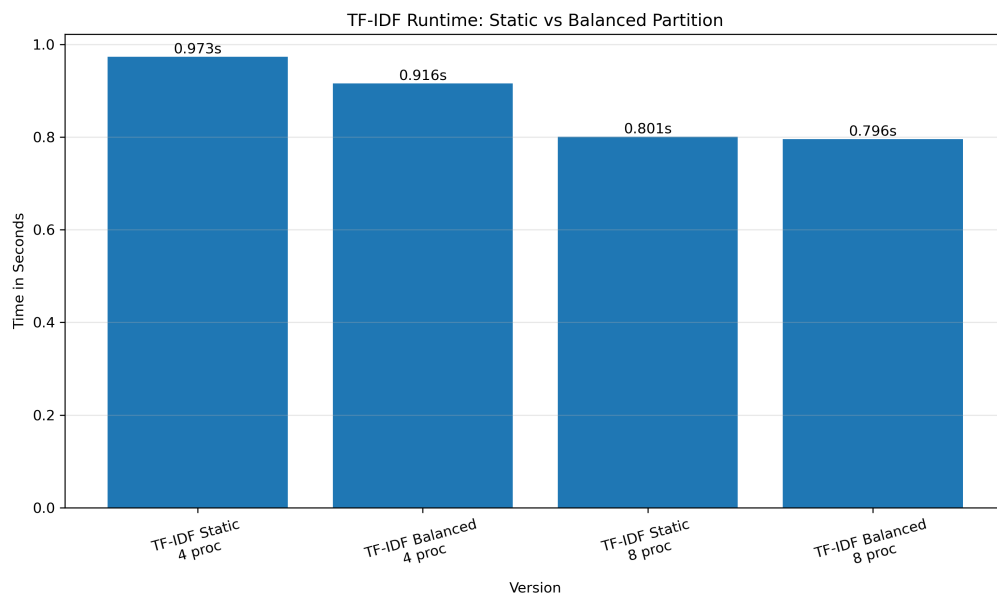


Figura 5: Tiempo de ejecución para TF-IDF con partición estática y balanceada.

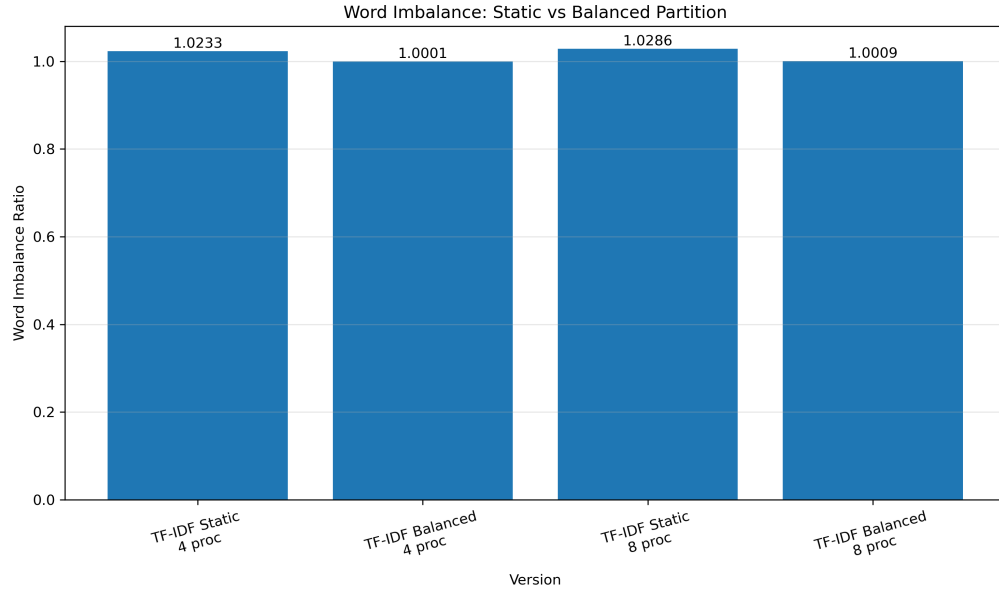


Figura 6: Desbalance de palabras para partición estática y balanceada.

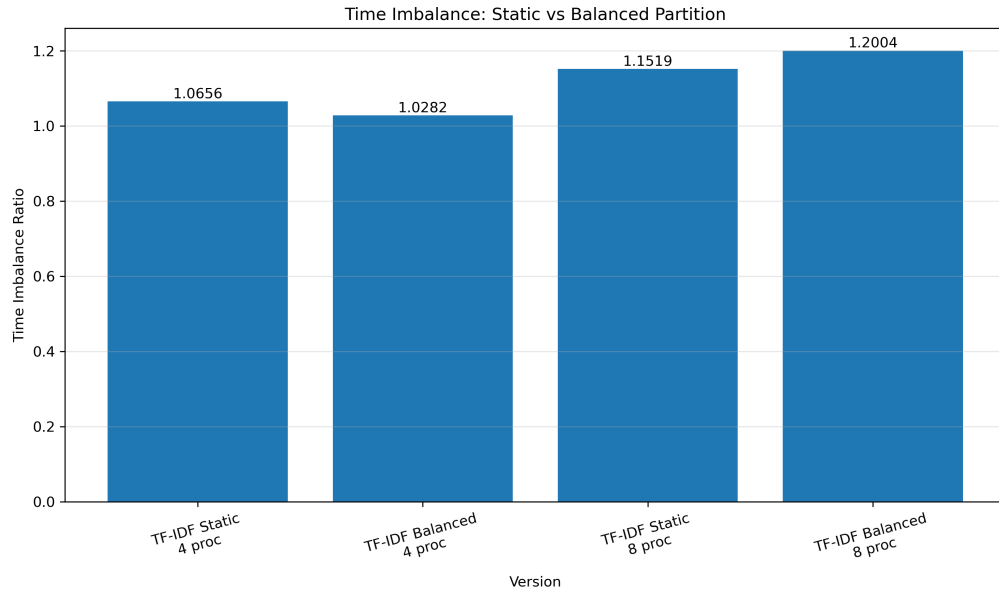


Figura 7: Desbalance de tiempo para partición estática y balanceada.

12. Discusión

Los resultados muestran que MPI permite reducir el tiempo de búsqueda al dividir el trabajo entre varios procesos. La versión MPI simple fue la más rápida porque usa un ranking básico y solo necesita una pasada por los documentos.

La versión MPI TF-IDF fue más costosa, pero también más completa. Esta versión calcula estadísticas globales de los términos usando MPI `Allreduce`. Esto agrega comunicación y una pasada adicional por el dataset, pero permite construir un ranking más realista.

La versión balanceada muestra que el diseño de la partición también importa. Repartir la misma cantidad de documentos no siempre es lo mismo que repartir la misma cantidad de trabajo. Al usar la cantidad de palabras como criterio, se logró una distribución más uniforme de palabras entre procesos.

Sin embargo, los resultados también muestran una limitación importante: un mejor balance de palabras no siempre produce un mejor balance de tiempo. Esto ocurre porque el tiempo real depende de más factores, como lectura de archivo, memoria, caché, actividad del sistema operativo y overhead de comunicación.

13. Limitaciones

Este proyecto usa un dataset sintético, no una colección real de documentos. Esto permite controlar el tamaño de los datos, pero no captura toda la complejidad de un motor de búsqueda real.

Además, el sistema no implementa un índice invertido completo. En un motor de búsqueda real, normalmente no se escanean todos los documentos en cada consulta. Se construye un índice para encontrar resultados más rápido.

14. Conclusión

En este proyecto se implementó un motor de búsqueda distribuido usando MPI. Se desarrollaron varias versiones para estudiar cómo el diseño afecta el rendimiento: serial, MPI simple, MPI TF-IDF y MPI TF-IDF balanceado por palabras.

Los resultados muestran que MPI reduce el tiempo de ejecución al aumentar el número de procesos. También muestran que una versión más avanzada, como TF-IDF, requiere más trabajo, pero produce un ranking más completo. Además, el balanceo por palabras ayuda a distribuir mejor la carga, aunque no garantiza tiempos perfectamente iguales.

El proyecto demuestra la importancia de medir el rendimiento con varias métricas. No basta con mirar el tiempo total; también es necesario analizar speedup, eficiencia y balance de carga para entender mejor el comportamiento de un programa paralelo.

15. Archivos principales

Los archivos principales del proyecto son:

- `src/search_serial.c`
- `src/search_mpi.c`
- `src/search_mpi_simple.c`
- `src/search_mpi_tfidf.c`
- `src/search_mpi_tfidf_balanced.c`
- `scripts/generate_documents.py`
- `scripts/plot_comparison.py`
- `scripts/plot_balance.py`